Grama Jyothi Rural Electricity Co-operative
Task 1

TASK 1 – Implement and Test the Billing Program (K3)

File to save as: Task01/billing.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225LL
#define THIRD_SLAB_PAISE 450LL
#define FIXED_CHARGE_PAISE 3000LL
#define METER_MAX 65535u
static uint32_t calculate_units(uint16_t previous, uint16_t current)
{
    /* Unsigned 16-bit subtraction gives the rollover consumption modulo 65536. */
    return (uint16_t)(current - previous);
}
static int64_t calculate_bill_paise(uint32_t units)
{
    int64_t energy = 0;
    if (units > FREE_UNITS) {
        uint32_t second = (units < SECOND_SLAB_LIMIT)
```

? units - FREE_UNITS

: SECOND_SLAB_LIMIT - FREE_UNITS;

```
            energy += (int64_t)second * SECOND_SLAB_PAISE;
        }
        if (units > SECOND_SLAB_LIMIT) {
            energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
        }
        return FIXED_CHARGE_PAISE + energy;
    }
    static void print_money(int64_t paise)
    {
        printf("Rs %" PRId64 ".%02" PRId64 "\n", paise / 100, paise % 100);
    }
    int main(void)
    {
        long long previous_input, current_input;
        uint16_t previous, current;
        uint32_t units;
        int64_t bill;
        printf("Enter previous reading (0-65535): ");
        if (scanf("%lld", &previous_input) != 1 ||
            previous_input < 0 || previous_input > METER_MAX) {
            fprintf(stderr, "Invalid previous reading.\n");
            return 1;
        }
        printf("Enter current reading (0-65535): ");
        if (scanf("%lld", &current_input) != 1 ||
            current_input < 0 || current_input > METER_MAX) {
            fprintf(stderr, "Invalid current reading.\n");
            return 1;
        }
        previous = (uint16_t)previous_input;
        current = (uint16_t)current_input;
        /* A decrease is treated as rollover only when it crosses the meter limit. */
        if (current < previous && previous < 65000u) {
            fprintf(stderr, "Invalid reading order: decrease is not an accepted rollover.\n");
            return 1;
        }
        units = calculate_units(previous, current);
        bill = calculate_bill_paise(units);
        printf("Units consumed: %" PRIu32 "\n", units);
        printf("Bill: ");
```

print_money(bill);

```
        return 0;
    }
```

Compile and Run

```
gcc -std=c11 -Wall -Wextra -pedantic Task01/billing.c -o Task01/billing
```

```
./Task01/billing
```

Note: The sample code accepts a lower current reading as rollover only when the previous reading is at least 65000. If your instructor expects a different rollover rule, document it and adjust the validation condition.

Test Plan and Results Run each case. Fill the Actual Result and Pass/Fail columns from your own program output. Important: The example code's rollover validation uses a practical threshold for demonstration. A 16-bit counter rollover is mathematically defined modulo 65536, but the program cannot always distinguish a genuine rollover from bad data using only two readings. State the rule you adopt and test it. Five-Bill Hand Calculation Use these sample consumption values to check the arithmetic; record your program's actual result. Expected total = Rs 937.50. Verify this by adding the five bills and by running your own test. Alternative: Arrays and a Loop

```
/* Conceptual alternative: slab boundaries and rates are stored in arrays.
   A production version must define clearly whether each rate applies to
   the whole slab or only units within that slab. */
const uint32_t upper[] = {100u, 200u, UINT32_MAX};
const int64_t rate_paise[] = {0LL, 225LL, 450LL};
/* Loop through slabs, calculate the units in each slab, and add
   slab_units * rate_paise[i]. */
```

With a fixed number of if statements, the calculation is O(1) time and O(1) extra space. An array-based loop over s slabs takes O(s) time and O(s) storage for the tariff arrays. The array design is easier to revise or extend to more slabs without adding many new conditional branches.